

НОВОСТИ
(+)[КОНТЕНТ](#) [WIKI](#)[MAN'ы](#) [ФОРУМ](#)Профиль: [Аноним](#) ([вход](#) | [регистрация](#))[Поиск](#) ([теги](#))[Каталог документации](#) / [Раздел "Программирование, языки"](#) / [Оглавление документа](#)

Advanced Bash-Scripting Guide: Искусство программирования на языке сценариев командной оболочки

[Назад](#)

Глава 12. Внешние команды, программы и утилиты

[Вперед](#)

12.4. Команды обработки текста

sort

Сортирует содержимое файла, часто используется как промежуточный фильтр в конвейерах. Эта команда сортирует поток текста в порядке убывания или возрастания, в зависимости от заданных опций. Ключ `-m` используется для сортировки и объединения входных файлов. В *странице info* перечислено большое количество возможных вариантов ключей. См. [Пример 10-9](#), [Пример 10-10](#) и [Пример A-9](#).

tsort

Топологическая сортировка, считывает пары строк, разделенных пробельными символами, и выполняет сортировку, в зависимости от заданного шаблона.

uniq

Удаляет повторяющиеся строки из отсортированного файла. Эту команду часто можно встретить в конвейере с командой [sort](#).

```
cat list-1 list-2 list-3 | sort | uniq > final.list
# Содержимое файлов,
# сортируется,
# затем удаляются повторяющиеся строки,
# и результат записывается в выходной файл.
```

Ключ `-c` выводит количество повторяющихся строк.

```
bash$ cat testfile
Эта строка встречается только один раз.
Эта строка встречается дважды.
Эта строка встречается дважды.
Эта строка встречается трижды.
Эта строка встречается трижды.
Эта строка встречается трижды.
```

```
bash$ uniq -c testfile
```

- 1 Эта строка встречается только один раз.
- 2 Эта строка встречается дважды.
- 3 Эта строка встречается трижды.

```
bash$ sort testfile | uniq -c | sort -nr
```

- 3 Эта строка встречается трижды.
- 2 Эта строка встречается дважды.
- 1 Эта строка встречается только один раз.

Команда `sort INPUTFILE | uniq -c | sort -nr` выводит *статистику встречаемости* строк в файле INPUTFILE (ключ `-nr`, в команде `sort`, означает сортировку в порядке убывания). Этот шаблон может с успехом использоваться при анализе файлов системного журнала, словарей и везде, где необходимо проанализировать лексическую структуру документа.

Пример 12-8. Частота встречаемости отдельных слов

```
#!/bin/bash
# wf.sh: "Сырой" анализ частоты встречаемости слова в текстовом файле.
```

```
ARGS=1
```

```
E_BADARGS=65
```

```
E_NOFILE=66
```

```
if [ $# -ne "$ARGS" ] # Файл для анализа задан?
then
    echo "Порядок использования: `basename $0` filename"
    exit $E_BADARGS
fi
```

```
if [ ! -f "$1" ]      # Проверка существования файла.
then
    echo "Файл \"$1\" не найден."
    exit $E_NOFILE
fi
```

```
#####
# main ()
sed -e 's/\././g' -e 's/ /\n/g' "$1" | tr 'A-Z' 'a-z' | sort | uniq -c | sort -nr
```

```
#
# =====
# Подсчет количества вхождений

# Точки и пробелы заменяются
#+ символами перевода строки,
#+ затем символы переводятся в нижний регистр
#+ и наконец подсчитывается количество вхождений,
#+ и выполняется сортировка по числу вхождений.
#####

# Упражнения:
# -----
# 1) Добавьте команду 'sed' для отсекаания других знаков пунктуации, например, запятых.
# 2) Добавьте удаление лишних пробелов и других пробельных символов.
# 3) Добавьте дополнительную сортировку так, чтобы слова с одинаковой частотой встречаемости
#+ сортировались бы в алфавитном порядке.

exit 0

bash$ cat testfile
Эта строка встречается только один раз.
Эта строка встречается дважды.
Эта строка встречается дважды.
Эта строка встречается трижды.
Эта строка встречается трижды.
Эта строка встречается трижды.

bash$ ./wf.sh testfile
6 Эта
6 встречается
6 строка
3 трижды
2 дважды
1 только
1 один
1 раз
```

expand, unexpand

Команда **expand** преобразует символы табуляции в пробелы. Часто используется в конвейерной обработке текста.

Команда **unexpand** преобразует пробелы в символы табуляции. Т.е. она является обратной по отношению к команде **expand**.

cut

Предназначена для извлечения отдельных полей из текстовых файлов. Напоминает команду **print \$N** в **awk**, но более ограничена в своих возможностях. В простейших случаях может быть неплохой заменой **awk** в сценариях. Особую значимость, для команды **cut**, представляют ключи **-d** (разделитель полей) и **-f** (номер(а) поля(ей)).

Использование команды **cut** для получения списка смонтированных файловых систем:

```
cat /etc/mtab | cut -d ' ' -f1,2
```

Использование команды **cut** для получения версии ОС и ядра:

```
uname -a | cut -d" " -f1,3,11,12
```

Использование команды **cut** для извлечения заголовков сообщений из электронных писем:

```
bash$ grep '^Subject:' read-messages | cut -c10-80
Re: Linux suitable for mission-critical apps?
MAKE MILLIONS WORKING AT HOME3
Spam complaint
Re: Spam complaint
```

Использование команды **cut** при разборе текстового файла:

```
# Список пользователей в /etc/passwd.
```

```
FILENAME=/etc/passwd
```

```
for user in $(cut -d: -f1 $FILENAME)
do
    echo $user
done
```

```
# Спасибо Oleg Philon за этот пример.
```

cut -d ' ' -f2,3 filename эквивалентно **awk -F'[]' '{ print \$2, \$3 }' filename**

См. также [Пример 12-33](#).

paste

Используется для объединения нескольких файлов в один многоколоночный файл.

join

Может рассматриваться как команда, родственная команде **paste**. Эта мощная утилита позволяет объединять два файла по общему полю, что представляет собой упрощенную версию реляционной базы данных.

Команда **join** оперирует только двумя файлами и объединяет только те строки, которые имеют общее поле (обычно числовое), результат объединения выводится на stdout. Объединяемые файлы должны быть отсортированы по ключевому полю.

File: 1.data

```
100 Shoes
200 Laces
300 Socks
```

File: 2.data

```
100 $40.00
200 $1.00
300 $2.00
```

```
bash$ join 1.data 2.data
```

File: 1.data 2.data

```
100 Shoes $40.00
200 Laces $1.00
300 Socks $2.00
```



На выходе ключевое поле встречается только один раз.

head

Выводит начальные строки из файла на stdout (по-умолчанию -- 10 строк, но это число можно задать иным). Эта команда имеет ряд интересных ключей.

Пример 12-9. Какие из файлов являются сценариями?

```
#!/bin/bash
# script-detector.sh: Отыскивает файлы сценариев в каталоге.

TESTCHARS=2    # Проверяются первые два символа.
SHABANG='#!'    # Сценарии как правило начинаются с "sha-bang."

for file in * # Обход всех файлов в каталоге.
do
    if [[ `head -c$TESTCHARS "$file"` = "$SHABANG" ]]
    #     head -c2                               #!
    # Ключ '-c' в команде "head" выводит заданное
    #+ количество символов, а не строк.
    then
        echo "Файл \"$file\" -- сценарий."
    else
        echo "Файл \"$file\" не является сценарием."
    fi
done

exit 0
```

Пример 12-10. Генератор 10-значных случайных чисел

```
#!/bin/bash
# rnd.sh: Генератор 10-значных случайных чисел

# Автор: Stephane Chazelas.

head -c4 /dev/urandom | od -N4 -tu4 | sed -ne '1s/.*/p'

# ===== #

# Описание
# -----

# head:
# -c4 -- первые 4 байта.

# od:
# -N4 ограничивает вывод 4-мя байтами.
# -tu4 беззнаковый десятичный формат вывода.

# sed:
# -n, в комбинации с флагом "p", в команде "s",
# выводит только совпадающие с шаблоном строки.
```

```
# Автор сценария описывает действия 'sed' таким образом:

# head -c4 /dev/urandom | od -N4 -tu4 | sed -ne '1s/.*/p'
# -----> |

# Передает вывод в "sed" -----> |
# пусть это будет 0000000 1198195154\n

# sed начинает читать символы: 0000000 1198195154\n.
# Здесь он находит символ перевода строки,
# таким образом он получает строку (0000000 1198195154).
# Затем он просматривает <диапазон><действие>. Первый и единственный -- это

# диапазон действие
# 1 s/.*/p

# Номер строки попадает в заданный диапазон, так что теперь он приступает к выполнению
действия:
# пытается заменить наибольшую подстроку, заканчивающуюся пробелом
# ("0000000 ") "ничем" (//), и если замена произведена -- выводит результат
# ("p" -- это флаг команды "s", а не команда "p", которая имеет иное значение).

# теперь sed готов продолжить чтение входного потока. (Обратите внимание:
# если опустить ключ -n, то sed выведет строку еще раз)

# Теперь sed дочитывает остаток строки.
# Он готов приступить к анализу 2-й строки (которая отмечена '$'
# как последняя).
# Поскольку строка не попадает в заданный <диапазон>, на этом обработка прекращается.

# Проще говоря, команда sed означает:
# "В первой строке удалить любые символы, вплоть до последнего встреченного пробела,
# и затем вывести остаток."

# Сделать это можно более простым способом:
# sed -e 's/.*/;;q'

# Где, заданы два <диапазона><действия> (можно записать и по другому
# sed -e 's/.*/ //' -e q):

# диапазон действие
# ничего (для совпадающих строк) s/.*/ //
# ничего (для совпадающих строк) q (quit)

# Здесь sed считывает только первую строку.
# Выполняет оба действия, и выводит строку перед завершением
```

(действие "q"), поскольку ключ "-n" опущен.

=====

Простая альтернатива:

head -c4 /dev/urandom| od -An -tu4

exit 0

См. также [Пример 12-30](#).

tail

Выводит последние строки из файла на stdout (по-умолчанию -- 10 строк). Обычно используется для мониторинга системных журналов. Ключ -f, позволяет вести непрерывное наблюдение за добавляемыми строками в файл.

Пример 12-11. Мониторинг системного журнала с помощью tail

#!/bin/bash

filename=sys.log

cat /dev/null > \$filename; echo "Создание / очистка временного файла."

Если файл отсутствует, то он создается,

#+ и очищается, если существует.

: > filename и > filename дают тот же эффект.

tail /var/log/messages > \$filename

Файл /var/log/messages должен быть доступен для чтения.

echo "В файл \$filename записаны последние строки из /var/log/messages."

exit 0

См. также [Пример 12-4](#), [Пример 12-30](#) и [Пример 29-6](#).

grep

Многоцелевая поисковая утилита, использующая [регулярные выражения](#). Изначально это была команда в древнем строчном редакторе **ed**, **g/re/p**, что означает -- *global - regular expression - print*.

grep pattern [file...]

Поиск участков текста в файле(ах), соответствующих шаблону *pattern*, где *pattern* может быть как обычной строкой, так и регулярным выражением.

bash\$ **grep '[rst]ystem.\$'** osinfo.txt

The GPL governs the distribution of the Linux operating system.

Если файл(ы) для поиска не задан, то команда **grep** работает как фильтр для устройства stdout, например в [конвейере](#).

```
bash$ ps ax | grep clock
765 tty1      S      0:00 xclock
901 pts/1     S      0:00 grep clock
```

-i -- выполняется поиск без учета регистра символов.

-w -- поиск совпадений целого слова.

-l -- вывод только имен файлов, в которых найдены участки, совпадающие с заданным образцом/шаблоном, без вывода совпадающих строк.

-r -- (рекурсивный поиск) поиск выполняется в текущем каталоге и всех вложенных подкаталогах.

The -n option lists the matching lines, together with line numbers.

```
bash$ grep -n Linux osinfo.txt
2:This is a file containing information about Linux.
6:The GPL governs the distribution of the Linux operating system.
```

-v (или --invert-match) -- выводит только строки, не содержащие совпадений.

```
grep pattern1 *.txt | grep -v pattern2
```

```
# Выводятся строки из "*.txt", совпадающие с "pattern1",
# но ***не*** совпадающие с "pattern2".
```

-c (--count) -- выводит количество совпадений без вывода самих совпадений.

```
grep -c txt *.sgml # (количество совпадений с "txt" в "*.sgml" файлах)
```

```
# grep -cz .
#           ^ точка
# означает подсчет (-c) непустых (". " -- содержащих хотя бы один символ) элементов,
# разделенных нулевыми байтами (-z)
#
printf 'a b\nc  d\n\n\n\n\n\000\n\000e\000\000\nf' | grep -cz .      # 4
printf 'a b\nc  d\n\n\n\n\n\000\n\000e\000\000\nf' | grep -cz '$'    # 5
```

```
printf 'a b\nc d\n\n\n\n\n\000\n\000e\000\000\nf' | grep -cz '^' # 5
#
printf 'a b\nc d\n\n\n\n\n\n\000\n\000e\000\000\nf' | grep -c '$' # 9
# По-умолчанию, в качестве разделителя, принимается символ перевода строки (\n).

# Обратите внимание: ключ -z характерен для GNU-версии "grep".

# Спасибо S.C.
```

Если **grep** вызывается для поиска по группе файлов, то вывод будет содержать указание на имена файлов, в которых найдены совпадения.

```
bash$ grep Linux osinfo.txt misc.txt
osinfo.txt:This is a file containing information about Linux.
osinfo.txt:The GPL governs the distribution of the Linux operating system.
misc.txt:The Linux operating system is steadily gaining in popularity.
```

 Для того, чтобы заставить **grep** выводить имя файла, когда поиск производится по одному-единственному файлу, достаточно указать устройство /dev/null в качестве второго файла.

```
bash$ grep Linux osinfo.txt /dev/null
osinfo.txt:This is a file containing information about Linux.
osinfo.txt:The GPL governs the distribution of the Linux operating system.
```

Если совпадение было найдено, то **grep** возвращает **код завершения** -- 0, это может оказаться полезным при выполнении поиска в условных операторах (в таких случаях особый интерес может представлять ключ -q, который подавляет вывод).

```
SUCCESS=0 # если найдено совпадение
word=Linux
filename=data.file

grep -q "$word" "$filename" # "-q" -- подавляет вывод на stdout.

if [ $? -eq $SUCCESS ]
then
    echo "Образец $word найден в $filename"
else
```

```
echo "Образец $word в файле $filename не найден"
fi
```

Пример 29-6 -- пример поиска заданного образца в системном журнале, с помощью **grep**.

Пример 12-12. Сценарий-эмулятор "грер"

```
#!/bin/bash
# grg.sh: Очень "грубая" реализация 'грер'.

E_BADARGS=65

if [ -z "$1" ]    # Проверка наличия аргументов.
then
    echo "Порядок использования: `basename $0` pattern"
    exit $E_BADARGS
fi

echo

for file in *      # Обход всех файлов в $PWD.
do
    output=$(sed -n /"$1"/p $file) # Подстановка команд.

    if [ ! -z "$output" ]          # Что произойдет, если кавычки вокруг "$output" убрать?
    then
        echo -n "$file: "
        echo $output
    fi
    # эквивалент: sed -ne "/$1/s|^|${file}: |p"

    echo
done

echo

exit 0

# Упражнения:
# -----
# 1) Добавьте вывод символов перевода строки, если найдено более одного совпадения в любом из
#    файлов.
# 2) Добавьте обработку различных ключей.
```



еггер -- то же самое, что и **грер -E**. Эта команда использует несколько отличающийся, расширенный набор **регулярных выражений**, что позволяет выполнять поиск более гибко.

fgrep -- то же самое, что и **grep -F**. Эта команда выполняет поиск строк символов (не регулярных выражений), что несколько увеличивает скорость поиска.

Утилита **agrep** имеет более широкие возможности поиска приблизительных совпадений. Образец поиска может отличаться от найденной строки на указанное число символов.



Для поиска по сжатым файлам следует использовать утилиты **zgrep**, **zegrep** или **zfgrep**. Они с успехом могут использоваться и для не сжатых файлов, но в этом случае они уступают в скорости обычным **grep**, **egrep** и **fgrep**. Они очень удобны при выполнении поиска по смешенному набору файлов -- когда одни файлы сжаты, а другие нет.

Для поиска по **bzip**-файлам используйте **bzgrep**.

look

Команда **look** очень похожа на **grep**, и предназначена для поиска по "словарям" -- отсортированным файлам. По-умолчанию, поиск выполняется в файле `/usr/dict/words`, но может быть указан и другой словарь.

Пример 12-13. Поиск слов в словаре

```
#!/bin/bash
# lookup: Выполняется поиск каждого слова из файла в словаре.

file=words.data # Файл с искомыми словами.

echo

while [ "$word" != end ] # Последнее слово в файле.
do
    read word          # Из файла, потому, что выполнено перенаправление в конце цикла.
    look $word > /dev/null # Подавление вывода строк из словаря.
    lookup=$?          # Код возврата команды 'look'.

    if [ "$lookup" -eq 0 ]
    then
        echo "Слово \"$word\" найдено."
    else
        echo "Слово \"$word\" не найдено."
    fi

done <"$file" # Перенаправление ввода из файла $file, так что "чтение" производится оттуда.

echo

exit 0

# -----
# Строки, расположенные ниже не будут исполнены, поскольку выше стоит команда "exit".
```

Stephane Chazelas предложил более короткий вариант:

```
while read word && [[ $word != end ]]
do if look "$word" > /dev/null
  then echo "Слово \"$word\" найдено."
  else echo "Слово \"$word\" не найдено."
  fi
done <"$file"

exit 0
```

sed, awk

Скриптовые языки, специально разработанные для анализа текстовых данных.

sed

Неинтерактивный "поточковый редактор". Широко используется в сценариях на языке командной оболочки.

awk

Утилита контекстного поиска и преобразования текста, замечательный инструмент для извлечения и/или обработки полей (колонок) в структурированных текстовых файлах. Синтаксис awk напоминает язык C.

WC

wc -- "word count", счетчик слов в файле или в потоке:

```
bash $ wc /usr/doc/sed-3.02/README
20      127      838 /usr/doc/sed-3.02/README
[20 строк  127 слов  838 символов]
```

wc -w подсчитывает только слова.

wc -l подсчитывает только строки.

wc -c подсчитывает только символы.

wc -L возвращает длину наибольшей строки.

Подсчет количества *.txt*-файлов в текущем каталоге с помощью **wc**:

```
$ ls *.txt | wc -l
```

Эта команда будет работать, если ни в одном из имен файлов "*.txt" нет символа перевода

строки.

```
# Альтернативный вариант:
# find . -maxdepth 1 -name \*.txt -print0 | grep -cz .
# (shopt -s nullglob; set -- *.txt; echo $#)

# Спасибо S.C.
```

Подсчет общего размера файлов, чьи имена начинаются с символов, в диапазоне d - h

```
bash$ wc [d-h]* | grep total | awk '{print $3}'
71832
```

От переводчика: в случае, если у вас локаль отлична от "C", то вышеприведенная команда может не дать результата, поскольку **wc** вернет не слово "total", в конце вывода, а "итого". Тогда можно попробовать несколько измененный вариант:

```
bash$ wc [d-h]* | grep итого | awk '{print $3}'
71832
```

Использование **wc** для подсчета количества вхождений слова "Linux" в основной исходный файл с текстом этого руководства.

```
bash$ grep Linux abs-book.sgml | wc -l
50
```

См. также [Пример 12-30](#) и [Пример 16-7](#).

Отдельные команды располагают функциональностью **wc** в виде своих ключей.

```
... | grep foo | wc -l
# Часто встречающаяся конструкция, которая может быть сокращена.

... | grep -c foo
# Ключ "-c" ("--count") команды grep.
```

Спасибо S.C.

tr

Замена одних символов на другие.



В отдельных случаях **символы необходимо заключать в кавычки и/или квадратные скобки**. Кавычки предотвращают интерпретацию специальных символов командной оболочкой. Квадратные скобки должны заключаться в кавычки.

Команда `tr "A-Z" "*" <filename` или `tr A-Z \* <filename` заменяет все символы верхнего регистра в `filename` на звездочки (вывод производится на `stdout`). В некоторых системах этот вариант может оказаться неработоспособным, тогда попробуйте `tr A-Z '["*"]'`.

Ключ `-d` удаляет символы из заданного диапазона.

```
echo "abcdef"           # abcdef
echo "abcdef" | tr -d b-d # aef
```

```
tr -d 0-9 <filename
# Удалит все цифровые символы из файла "filename".
```

Ключ `--squeeze-repeats (-s)` удалит все повторяющиеся последовательности символов. Может использоваться для удаления лишних **пробельных символов**.

```
bash$ echo "XXXXX" | tr --squeeze-repeats 'X'
X
```

Ключ `-c "complement"` *заменит* символы в соответствии с шаблоном. Этот ключ воздействует только на те символы, которые НЕ соответствуют заданному шаблону.

```
bash$ echo "acfdeb123" | tr -c b-d +
+c+d+b++++
```

Обратите внимание: команда `tr` корректно распознает **символьные классы POSIX**. [1]

```
bash$ echo "abcd2ef1" | tr '[:alpha:]' -
----2--1
```

Пример 12-14. toupper: Преобразование символов в верхний регистр.

```
#!/bin/bash
# Преобразование символов в верхний регистр.

E_BADARGS=65

if [ -z "$1" ] # Стандартная проверка командной строки.
then
    echo "Порядок использования: `basename $0` filename"
    exit $E_BADARGS
fi

tr a-z A-Z <"$1"

# Тот же эффект можно получить при использовании символьных классов POSIX:
#      tr '[:lower:]' '[:upper:]' <"$1"
# Спасибо S.C.

exit 0
```

Пример 12-15. lowercase: Изменение имен всех файлов в текущем каталоге в нижний регистр.

```
#!/bin/bash
#
# Изменит все имена файлов в текущем каталоге в нижний регистр.
#

for filename in *          # Обход всех файлов в каталоге.
do
    fname=`basename $filename`
    n=`echo $fname | tr A-Z a-z` # Перевести символы в нижний регистр.
    if [ "$fname" != "$n" ]      # Переименовать только те файлы, имена которых изменились.
    then
        mv $fname $n
    fi
done

exit 0

# Сроки приведенные ниже не будут исполняться, поскольку выше стоит команда "exit".
#-----#
# Запустите эту часть сценария, удалив строки , стоящие выше.

# Сценарий, приведенный выше, не работает с именами файлов, содержащими пробелы или символы
перевода строки.

# В связи с этим, Stephane Chazelas предложил следующий вариант:
```

```

for filename in *      # Нет необходимости использовать basename,
                        # поскольку "*" возвращает имена, не содержащие "/".
do n=`echo "$filename/" | tr '[:upper:]' '[:lower:]'`
#                      символные классы POSIX.
#                      Завершающий слэш добавлен для того, чтобы символ перевода строки
#                      не был удален при подстановке команды.
# Подстановка переменной:
n=${n%/}              # Удаление завершающего слэша, добавленного выше.
[[ $filename == $n ]] || mv "$filename" "$n"
                        # Проверка -- действительно ли изменилось имя файла.
done

exit 0

```

Пример 12-16. du: Преобразование текстового файла из формата DOS в формат UNIX.

```

#!/bin/bash
# du.sh: Преобразование текстового файла из формата DOS в формат UNIX.

E_WRONGARGS=65

if [ -z "$1" ]
then
    echo "Порядок использования: `basename $0` filename-to-convert"
    exit $E_WRONGARGS
fi

NEWFILENAME=$1.unx

CR='\015' # Возврат каретки.
# Строки в текстовых файлах DOS завершаются комбинацией символов CR-LF.

tr -d $CR < $1 > $NEWFILENAME
# Удалить символы CR и записать в новый файл.

echo "Исходный текстовый файл: \"$1\"."
echo "Преобразованный файл: \"$NEWFILENAME\"."

exit 0

```

Пример 12-17. rot13: Сверхслабое шифрование по алгоритму rot13.

```

#!/bin/bash
# rot13.sh: Классический алгоритм шифрования rot13,
#           который способен "расколоть" даже 3-х летний ребенок.

# Порядок использования: ./rot13.sh filename

```

```
# или ./rot13.sh <filename
# или ./rot13.sh и ввести текст с клавиатуры (stdin)

cat "$@" | tr 'a-zA-Z' 'n-za-mN-ZA-M' # "a" заменяется на "n", "b" на "o", и т.д.
# Конструкция 'cat "$@"'
#+ позволяет вводить данные как со stdin, так и из файла.

exit 0
```

Пример 12-18. Более "сложный" шифр

```
#!/bin/bash
# crypto-quote.sh: Ограниченное шифрование

# Шифрование ограничивается простой заменой одних алфавитных символов другими.
# Результат очень похож на шифры-загадки

key=ETAOINSHRDLUBCFGJMQPVWZYXK
# Здесь, "key" -- ни что иное, как "перемешанный" алфавит.
# Изменение ключа "key" приведет к изменению шифра.

# Конструкция 'cat "$@"' позволяет вводить данные как со stdin, так и из файла.
# Если используется stdin, то ввод должен завершаться комбинацией Control-D.
# Иначе, в командной строке, сценарию должно быть передано имя файла.

cat "$@" | tr "a-z" "A-Z" | tr "A-Z" "$key"
# | в верхний регистр | шифрование
# Такой прием позволяет шифровать как символы в верхнем регистре, так и в нижнем.
# Неалфавитные символы остаются без изменений.

# Попробуйте зашифровать какой либо текст, например
# "Nothing so needs reforming as other people's habits."
# --Mark Twain
#
# Результат будет:
# "CFPHRCS QF CIIQ MINFMBRCS EQ FPHIM GIFGUI'Q HETRPQ."
# --BEML PZERC

# Для дешифрации можно использовать следующую комбинацию:
# cat "$@" | tr "$key" "A-Z"

# Этот нехитрый шифр может быть "взломан" 12-ти летним ребенком
#+ с помощью карандаша и бумаги.

exit 0
```

Различные версии `tr`

Утилита `tr` имеет две, исторически сложившиеся, версии. BSD-версия не использует квадратные скобки (`tr a-z A-Z`), в то время как SysV-версия использует их (`tr '[a-z]' '[A-Z]'`). GNU-версия утилиты `tr` напоминает версию BSD, но диапазоны символов обязательно должны заключаться в квадратные скобки.

fold

Выравнивает текст по ширине, разрывая, если это необходимо, слова. Особый интерес представляет ключ `-s`, который производит перенос строк по пробелам, стараясь не разрывать слова. (см. [Пример 12-19](#) и [Пример A-2](#)).

fmt

Очень простая утилита форматирования текста, чаще всего используемая как фильтр в конвейерах для того, чтобы выполнить "перенос" длинных строк текста.

Пример 12-19. Отформатированный список файлов.

```
#!/bin/bash

WIDTH=40                # 40 символов в строке.

b=`ls /usr/local/bin`    # Получить список файлов...

echo $b | fmt -w $WIDTH

# То же самое можно выполнить командой
# echo $b | fold - -s -w $WIDTH

exit 0
```

См. также [Пример 12-4](#).



Очень мощной альтернативой утилите `fmt`, является утилита `par` (автор Kamil Toman), которую вы сможете найти на <http://www.cs.berkeley.edu/~amc/Par/>.

col

Эта утилита с обманчивым названием удаляет из входного потока символы обратной подачи бумаги (код ESC 7). Она так же пытается заменить пробелы на табуляции. Основная область применения утилиты `col --` фильтрация вывода отдельных утилит обработки текста, таких как `groff` и `tbl`.

column

Форматирование по столбцам. Эта утилита преобразует текст, например какой либо список, в табличное, более "удобочитаемое", представляя, вставляя символы табуляции по мере необходимости.

Пример 12-20. Пример форматирования списка файлов в каталоге

```
#!/bin/bash
# За основу сценария взят пример "map column".

(printf "PERMISSIONS LINKS OWNER GROUP SIZE DATE TIME PROG-NAME\n" \
; ls -l | sed 1d) | column -t

# Команда "sed 1d" удаляет первую строку, выводимую командой ls,
#+ (для локали "C" это строка: "total      N",
#+ где "N" -- общее количество файлов.

# Ключ -t, команды "column", означает "табличное" представление.

exit 0
```

colrm

Утилита удаления колонок. Удаляет колонки (столбцы) символов из файла и выводит результат на stdout. **colrm 2 4 <filename** -- удалит символы со 2-го по 4-й включительно, в каждой строке в файле filename.



Если файл содержит символы табуляции или непечатаемые символы, то результат может получиться самым неожиданным. В таких случаях, как правило, утилиту **colrm**, в конвейере, окружают командами **expand** и **unexpand**.

nl

Нумерует строки в файле. **nl filename** -- выведет файл filename на stdout, и в начале каждой строки вставит ее порядковый номер, счет начинается с первой непустой строки. Если файл не указывается, то принимается ввод со stdin.

Вывод команды **nl** очень напоминает **cat -n**, однако, по-умолчанию **nl** не нумерует пустые строки.

Пример 12-21. nl: Самонумерующийся сценарий.

```
#!/bin/bash

# Сценарий выводит себя сам на stdout дважды, нумеруя строки сценария.

# 'nl' вставит для этой строки номер 3, поскольку она не нумерует пустые строки.
# 'cat -n' вставит для этой строки номер 5.

nl `basename $0`

echo; echo # А теперь попробуем вывести текст сценария с помощью 'cat -n'

cat -n `basename $0`
# Различия состоят в том, что 'cat -n' нумерует все строки.
# Обратите внимание: 'nl -ba' -- сделает то же самое.
```

`exit 0`

pr

Подготовка файла к печати. Утилита производит разбивку файла на страницы, приводя его в вид пригодный для печати или для вывода на экран. Разнообразные ключи позволяют выполнять различные манипуляции над строками и колонками, соединять строки, устанавливать поля, нумеровать строки, добавлять колонтитулы и многое, многое другое. Утилита **pr** соединяет в себе функциональность таких команд, как **nl**, **paste**, **fold**, **column** и **expand**.

`pr -o 5 --width=65 fileZZZ | more` -- выдаст хорошо оформленное и разбитое на страницы содержимое файла `fileZZZ`.

Хочу особо отметить ключ `-d`, который выводит строки с двойным интервалом (тот же эффект, что и `sed -G`).

gettext

GNU утилита, предназначена для нужд [локализации](#) и перевода сообщений программ, выводимых на экран, на язык пользователя. Не смотря на то, что это актуально, прежде всего, для программ на языке C, тем не менее **gettext** с успехом может использоваться в сценариях командной оболочки для тех же целей. См. *info page*.

iconv

Утилита преобразования текста из одной кодировки в другую. В основном используется для нужд локализации.

recode

Может рассматриваться как разновидность утилиты **iconv**, описанной выше. Универсальная утилита для преобразования текстовой информации в различные кодировки.

TeX, gs

TeX и **Postscript** -- языки разметки текста, используемые для подготовки текста к печати или выводу на экран.

TeX -- это сложная система подготовки к печати, разработанная Дональдом Кнудом (Donald Knuth). Эту утилиту удобнее использовать внутри сценария, чем в командной строке, поскольку в сценарии проще один раз записать все необходимые параметры, передаваемые утилите, для получения необходимого результата.

Ghostscript (**gs**) -- это GPL-версия интерпретатора Postscript.

groff, tbl, eqn

groff -- это еще один язык разметки текста и форматированного вывода. Является расширенной GNU-версией пакета **roff/troff** в UNIX-системах.

tbl -- утилита обработки таблиц, должна рассматриваться как составная часть **groff**, так как ее задачей является преобразование таблиц в команды **groff**.

eqn -- утилита преобразования математических выражений в команды **groff**.

lex, yacc

lex -- утилита лексического разбора текста. В Linux-системах заменена на свободно распространяемую утилиту **flex**.

yacc -- утилита для создания синтаксических анализаторов, на основе набора грамматик, задаваемых разработчиком. В Linux-системах, эта утилита заменена на свободно распространяемую утилиту **bison**.

Примечания

- [1] Это верно только для GNU-версии команды **tr**, поведение этой команды, в коммерческих UNIX-системах, может несколько отличаться.

[Назад](#)

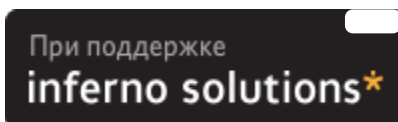
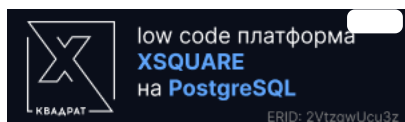
Команды для работы с датой и временем

[К началу](#)

[Наверх](#)

[Вперед](#)

Команды для работы с файлами и архивами



Хостинг:



Hoster.ru
хостинг провайдер

[Закладки на сайте](#)

[Проследить за страницей](#)

Created 1996-2026 by [Maxim Chirkov](#)

[Добавить новость](#), [Поддержать](#)